

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Алгоритмы и структуры данных

Лабораторная работа № 1

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлунин Александр Александрович

г. Санкт-Петербург 2026 г

Цель:.....	4
Задача №1. Максимальная стоимость добычи (Дробный рюкзак).....	4
Задача №2. Заправки.....	6
Задача №3. Максимальное скалярное произведение.....	7
Задача №7. Проблема сапожника.....	8
Задача №8. Расписание лекций.....	9
Задача №9. Распечатка.....	11
Задача №10. Яблоки.....	12
Задача №12. Разбиение на 2 кучи.....	16
Задача №13. Разбиение на 3 кучи.....	18
Задача №14. Максимальное выражение.....	19
Задача №15. Скобочная последовательность.....	21
Задача №17. Ход конем.....	23
Задача №18. Кафе.....	25
Вывод по работе.....	27

Цель:

Целью данной лабораторной работы является изучение и практическое применение жадных алгоритмов и методов динамического программирования при решении задач различной сложности. В ходе выполнения работы были реализованы алгоритмы для задач на оптимальный выбор, разбиение на подзадачи, восстановление ответа и минимизацию затрат.

В соответствии с вариантом 21 были решены основные задачи №1, №7, №9, №17 и №18. Дополнительно были выполнены задачи №2, №3, №8, №10, №11, №12, №13, №14 и №15

Задача №1. Максимальная стоимость добычи (Дробный рюкзак)

Текст задачи:

Вор находит гораздо больше добычи, чем может поместиться в его сумке. Помогите ему найти самую ценную комбинацию предметов, предполагая, что любая часть предмета добычи может быть помещена в его сумку. Цель - реализовать алгоритм для задачи о дробном рюкзаке.

Код:

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    if not data:
        return

    n = int(data[0])
    W = int(data[1])

    items = []
    idx = 2
    for _ in range(n):
        p = int(data[idx])
        w = int(data[idx + 1])
        idx += 2
        if w > 0:
            items.append((p / w, p, w))
        else:
            if p > 0:
                items.append((float("inf"), p, w))

    items.sort(reverse=True)

    ans = 0.0
    capacity = W

    for ratio, price, weight in items:
        if capacity == 0:
            break

        if weight == 0:
```

```

        ans += price
        continue

    take = min(capacity, weight)
    ans += take * ratio
    capacity -= take

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(f"{ans:.4f}\n")

solve()

```

Текстовое объяснение решения: Применяется жадный алгоритм. Вычисляется удельная стоимость каждого предмета (цена за единицу веса). Предметы сортируются по убыванию этой стоимости. Затем рюкзак заполняется самыми выгодными предметами целиком, пока хватает места, а последний предмет берется дробно. Учтены краевые случаи с нулевым весом.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	3 50 / 60 20 / 100 50 / 120 30	180.0000
< 1 мс	< 1 Мб	1 10 / 500 30	166.6667
< 1 мс	< 1 Мб	2 100 / 10 10 / 20 20	30.0000

Задача №2. Заправки

Текст задачи:

Автомобиль должен проехать расстояние d . Полный бак позволяет проехать m . Известны расстояния до заправок. Требуется определить минимальное число остановок или вывести -1, если путь невозможен.

Код:

```

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    d = data[0]
    m = data[1]
    n = data[2]
    stops = data[3:3 + n]

    stops = [0] + stops + [d]

    count = 0
    cur = 0

    while cur < len(stops) - 1:
        last = cur

        while cur < len(stops) - 1 and stops[cur + 1] - stops[last] <= m:
            cur += 1

        if cur == last:
            with open("output.txt", "w", encoding="utf-8") as f:
                f.write("-1\n")
            return

        if cur < len(stops) - 1:
            count += 1

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(count) + "\n")

solve()

```

Текстовое объяснение решения: Жадный алгоритм. На каждой станции мы смотрим максимально далеко вперед и едем до последней заправки, до которой хватает запаса хода m . Добавление старта 0 и финиша d в массив станций позволяет избежать дополнительных проверок выхода за границы маршрута.

Таблица тестирования:

время	память	input	output
-------	--------	-------	--------

< 1 мс	< 1 Мб	950 / 400 / 4 / 200 375 550 750	2
< 1 мс	< 1 Мб	10 / 3 / 4 / 1 2 5 9	-1
< 1 мс	< 1 Мб	200 / 250 / 2 / 100 150	0
< 1 мс	< 1 Мб	500 / 200 / 3 / 100 200 300	2

Задача №3. Максимальное скалярное произведение

Текст задачи:

У вас есть n объявлений для размещения на популярной интернет-странице. Для каждого объявления вы знаете, сколько рекламодатель готов платить за один клик по этому объявлению. Вы настроили n слотов на своей странице и оценили ожидаемое количество кликов в день для каждого слота. Теперь ваша цель - распределить рекламу по слотам, чтобы максимизировать общий доход.

Код:

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    a = data[1:1 + n]
    b = data[1 + n:1 + 2 * n]

    a.sort()
    b.sort()

    ans = 0
    for i in range(n):
        ans += a[i] * b[i]

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(ans) + "\n")

solve()
```

Текстовое объяснение решения: Чтобы получить максимальное скалярное произведение двух векторов, необходимо перемножать наибольшие элементы с наибольшими, а наименьшие с наименьшими (перемножение двух отрицательных чисел дает положительный результат). Для этого оба массива сортируются по возрастанию.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	1 / 23 / 39	897
< 1 мс	< 1 Мб	3 / 1 3 -5 / -2 4 1	23
< 1 мс	< 1 Мб	4 / 1 2 3 4 / 5 6 7 8	70
< 1 мс	< 1 Мб	3 / -1 -2 -3 / -1 -2 -3	14

Задача №7. Проблема сапожника

Текст задачи:

В некоей воинской части есть сапожник. Рабочий день сапожника длится К минут. Заведующий складом оценивает работу сапожника по количеству починенной обуви, независимо от того, насколько сложный ремонт требовался в каждом случае. Дано n сапог, нуждающихся в починке. Определите, какое максимальное количество из них сапожник сможет починить за один рабочий день.

Код:

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    if not data:
        return

    K = data[0]
    n = data[1]
    times = data[2:2 + n]

    times.sort()
```

```

total = 0
count = 0

for t in times:
    if total + t <= K:
        total += t
        count += 1
    else:
        break

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(count) + "\n")

solve()

```

Текстовое объяснение решения: Жадный выбор. Чтобы починить максимальное количество обуви, необходимо отдавать приоритет самым быстрым заказам. Массив времени сортируется по возрастанию, после чего заказы берутся в работу, пока суммарное время не превысит рабочий день K.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	10 3 / 6 2 8	2
< 1 мс	< 1 Мб	3 2 / 10 20	0
< 1 мс	< 1 Мб	15 4 / 3 3 3 3	4

Задача №8. Расписание лекций

Текст задачи:

Вы наверно знаете, что в ИТМО лекции читают одни из лучших преподаватели мира. К сожалению, лекционных аудиторий у нас не так уж и много, особенно на Биржевой, поэтому каждый преподаватель составил список лекций, которые он хочет прочитать студентам. Чтобы студенты, в начале февраля, увидели расписание лекций, необходимо его составить прямо сейчас. И без вас нам здесь не справиться. У нас есть список заявок от преподавателей на лекции для одной из аудиторий. Каждая заявка

представлена в виде временного интервала $[s_i, f_i)$ - время начала и конца лекции. Лекция считается открытым интервалом, то есть какая-то лекция может начаться в момент окончания другой, без перерыва. Необходимо выбрать из этих заявок такое подмножество, чтобы суммарно выполнить максимальное количество заявок. Учтите, что одновременно в лекционной аудитории, конечно же, может читаться лишь одна лекция

Код:

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    lectures = []

    idx = 1
    for _ in range(n):
        s = data[idx]
        t = data[idx + 1]
        idx += 2
        lectures.append((t, s))

    lectures.sort()

    count = 0
    last_end = 0

    for end, start in lectures:
        if start >= last_end:
            count += 1
            last_end = end

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(count) + "\n")

solve()
```

Текстовое объяснение решения: Классическая задача Activity Selection. Список заявок сортируется по времени их завершения. Выбирается лекция, которая заканчивается раньше всего, чтобы оставить максимум времени для следующих занятий.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	1 / 23 / 39	897
< 1 мс	< 1 Мб	3 / 1 3 -5 / -2 4 1	23
< 1 мс	< 1 Мб	4 / 1 2 3 4 / 5 6 7 8	70
< 1 мс	< 1 Мб	3 / -1 -2 -3 / -1 -2 -3	14

Задача №9. Распечатка

Текст задачи:

Диссертация дело сложное, особенно когда нужно ее печатать. При этом вам нужно распечатать не только текст самой диссертации, так и другие материалы (задание, рецензии, отзывы, авторефераты для защиты и т.п.). Вы оценили объём печати в N листов. Фирма, готовая размножить печатные материалы, предлагает следующие финансовые условия. Один лист она печатает за A1 рублей, 10 листов - за A2 рублей, 100 листов - за A3 рублей, 1000 листов - за A4 рублей, 10000 листов - за A5 рублей, 100000 листов - за A6 рублей, 1000000 листов - за A7 рублей. При этом не гарантируется, что один лист в более крупном заказе обойдется дешевле, чем в более мелком. И даже может оказаться, что для любой партии будет выгодно воспользоваться тарифом для одного листа. Печать конкретного заказа производится или путем комбинации нескольких тарифов, или путем заказа более крупной партии. Например, 980 листов можно распечатать, заказав печать 9 партий по 100 листов плюс 8 партий по 10 листов, сделав 98 заказов по 10 листов, 980 заказов по 1 листу или заказав печать 1000 (или даже 10000 и более) листов, если это окажется выгоднее. Требуется по заданному объему заказа в листах N определить минимальную сумму денег в рублях, которой будет достаточно для выполнения заказа.

Листинг кода:

```
def solve():  
    with open("input.txt", "r", encoding="utf-8") as f:
```

```

data = list(map(int, f.read().split()))

if not data:
    return

N = data[0]
a = data[1:8]

powers = [1, 10, 100, 1000, 10000, 100000, 1000000]

for i in range(1, 7):
    a[i] = min(a[i], 10 * a[i - 1])

ans = 10**30
cur_cost = 0
remaining = N

for i in range(6, -1, -1):
    pack_size = powers[i]
    cnt = remaining // pack_size

    cur_cost += cnt * a[i]
    remaining %= pack_size

    if remaining > 0:
        ans = min(ans, cur_cost + a[i])
    else:
        ans = min(ans, cur_cost)

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(ans) + "\n")

solve()

```

Текстовое объяснение решения: Производится нормализация тарифов, чтобы крупный пакет не стоил дороже набора мелких. Жадным алгоритмом заказ закрывается крупными пакетами. На каждой итерации проверяется возможность купить лишний крупный пакет для закрытия остатка, если это окажется дешевле точного набора мелкими.

Таблица тестирования:

время	память	input	output
-------	--------	-------	--------

< 1 мс	< 1 Мб	980 / 1 / 9 / 90 / 900 / 1000 / 10000 / 10000	180.0000
< 1 мс	< 1 Мб	980 / 1 / 10 / 100 / 1000 / 900 / 10000 / 10000	166.6667
< 1 мс	< 1 Мб	5 / 2 / 20 / 200 / 2000 / 20000 / 200000 / 2000000	30.0000

Задача №10. Яблоки

Текст задачи:

Алисе в стране чудес попались n волшебных яблок. Про каждое яблоко известно, что после того, как его съешь, твой рост сначала уменьшится на a_i сантиметров, а потом увеличится на b_i сантиметров. Алиса очень голодная и хочет съесть все n яблок, но боится, что в какой-то момент ее рост s станет равным нулю или еще меньше, и она пропадет совсем. Помогите ей узнать, можно ли съесть яблоки в таком порядке, чтобы в любой момент времени рост Алисы был больше нуля.

Код:

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    s = data[1]

    apples = []
    idx = 2
    for i in range(n):
        a = data[idx]
        b = data[idx + 1]
        idx += 2
        apples.append((a, b, i + 1))

    good = []
    bad = []

    for a, b, num in apples:
        if b - a >= 0:
```

```

        good.append((a, b, num))
    else:
        bad.append((b, a, num))

good.sort(key=lambda x: x[0])
bad.sort(reverse=True)

order = good + [(a, b, num) for b, a, num in bad]

ans = []
height = s

for a, b, num in order:
    if height - a <= 0:
        with open("output.txt", "w", encoding="utf-8") as f:
            f.write("-1\n")
        return
    height = height - a + b
    ans.append(num)

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(map(str, ans)) + "\n")

solve()

```

Текстовое объяснение решения: Все элементы делятся на «хорошие» (восстанавливают $\geq$ урона) и «плохие» (восстанавливают меньше урона). Хорошие проходятся в первую очередь и сортируются по возрастанию требуемого здоровья, чтобы выжить на ранних этапах. Плохие сортируются по убыванию восстанавливаемого здоровья, чтобы самые невыгодные отложить на конец, когда запас прочности максимален.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	3 5 2 3 10 5 5 10	1 3 2
< 1 мс	< 1 Мб	3 5 2 3	-1

		10 5 5 6	
< 1 мс	< 1 Мб	2 10 1 2 2 3	1 2
< 1 мс	< 1 Мб	1 3 3 5	-1

Задача №11. Рюкзак (0/1)

Текст задачи:

Вам дается набор золотых слитков, и ваша цель - набрать как можно больше золота в свою сумку. Существует только одна копия каждого слитка, и для каждого слитка вы можете либо взять его, либо нет (т.е. вы не можете взять часть слитка).

Код:

```
def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = list(map(int, f.read().split()))

        W = data[0]

        n = data[1]

        weights = data[2:2 + n]

        dp = [0] * (W + 1)

        for w in weights:

            for x in range(W, w - 1, -1):

                dp[x] = max(dp[x], dp[x - w] + w)

        with open("output.txt", "w", encoding="utf-8") as f:
```

```
f.write(str(dp[W]) + "\n")
```

```
solve()
```

Текстовое объяснение решения: Задача о дискретном рюкзаке решается методом одномерного динамического программирования. Перебираются все слитки, и для каждого обновляется массив `dp` строго справа налево (от вместимости W до веса слитка). Проход в обратном направлении гарантирует, что каждый слиток будет использован только один раз.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	10 3 1 4 8	9
< 1 мс	< 1 Мб	20 4 5 7 12 18	19
< 1 мс	< 1 Мб	15 5 6 3 4 2 10	15
< 1 мс	< 1 Мб	5 3 6 7 8	0

Задача №12. Разбиение на 2 кучи

Текст задачи:

Дана последовательность натуральных чисел $a_1, a_2, \dots, a_n$, и известно, что $a_i \leq i$ для любого $1 \leq i \leq n$. Требуется определить, можно ли разбить элементы последовательности на две части таким образом, что сумма элементов в каждой из частей будет равна половине суммы всех элементов последовательности.

Код:

```
def solve():
```

```

with open("input.txt", "r", encoding="utf-8") as f:
    data = list(map(int, f.read().split()))

n = data[0]
a = data[1:1 + n]

total = sum(a)

if total % 2 != 0:
    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("-1\n")
    return

target = total // 2

dp = [-1] * (target + 1)
dp[0] = 0

prev = [-1] * (target + 1)
take = [-1] * (target + 1)

for i in range(n):
    val = a[i]
    for s in range(target, val - 1, -1):
        if dp[s] == -1 and dp[s - val] != -1:
            dp[s] = 1
            prev[s] = s - val
            take[s] = i

if dp[target] == -1:
    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("-1\n")
    return

indices = []
cur = target
while cur > 0:
    indices.append(take[cur] + 1)
    cur = prev[cur]

indices.reverse()

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(len(indices)) + "\n")
    f.write(" ".join(map(str, indices)) + "\n")

```

solve()

Текстовое объяснение решения: Задача о сумме подмножеств. Цель — набрать ровно половину от общей суммы всех элементов. Реализовано одномерным ДП (похожим на рюкзак) с дополнительными массивами `prev` и `take` для возможности обратного прохода и восстановления индексов выбранных элементов.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	3 1 2 3	1 3
< 1 мс	< 1 Мб	4 1 1 2 2	2 1 4
< 1 мс	< 1 Мб	3 1 1 1	-1
< 1 мс	< 1 Мб	2 1 1	1 1

Задача №13. Разбиение на 3 кучи

Текст задачи:

Вы и двое ваших друзей только что вернулись домой после посещения разных стран. Теперь вы хотели бы поровну разделить все сувениры, которые все трое купили.

Листинг кода:

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    n = data[0]
    arr = data[1:1 + n]

    total = sum(arr)
```

```

if total % 3 != 0:
    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("0\n")
    return

```

```

target = total // 3

```

```

dp = [[False] * (target + 1) for _ in range(target + 1)]
dp[0][0] = True

```

```

for x in arr:
    new_dp = [row[:] for row in dp]
    for i in range(target + 1):
        for j in range(target + 1):
            if dp[i][j]:
                if i + x <= target:
                    new_dp[i + x][j] = True
                if j + x <= target:
                    new_dp[i][j + x] = True
    dp = new_dp

```

```

with open("output.txt", "w", encoding="utf-8") as f:
    if dp[target][target]:
        f.write("1\n")
    else:
        f.write("0\n")

```

```

solve()

```

Текстовое объяснение решения: Двумерное ДП. Состояние $dp[i][j]$ означает возможность собрать первую кучу с суммой i , а вторую с суммой j . Если возможно собрать две кучи с суммой $total/3$, то оставшиеся элементы гарантированно образуют третью кучу с такой же суммой.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	4 3 3 3 3	0
< 1 мс	< 1 Мб	1 40	0

< 1 мс	< 1 Мб	11 17 59 34 57 17 23 67 1 18 2 59	1
< 1 мс	< 1 Мб	6 1 1 1 1 1 1	1

Задача №14. Максимальное выражение

Текст задачи:

В этой задаче ваша цель - добавить скобки к заданному арифметическому выражению, чтобы максимизировать его значение. $\max(5 - 8 + 7 \times 4 - 8 + 9) = ?$

Код:

```
def eval_op(a, b, op):
    if op == "+":
        return a + b
    if op == "-":
        return a - b
    return a * b

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        s = f.read().strip()

    digits = []
    ops = []

    for i, ch in enumerate(s):
        if i % 2 == 0:
            digits.append(int(ch))
        else:
            ops.append(ch)

    n = len(digits)

    mn = [[0] * n for _ in range(n)]
    mx = [[0] * n for _ in range(n)]

    for i in range(n):
```

```

mn[i][i] = digits[i]
mx[i][i] = digits[i]

for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        min_val = 10**18
        max_val = -10**18

        for k in range(i, j):
            a = eval_op(mx[i][k], mx[k + 1][j], ops[k])
            b = eval_op(mx[i][k], mn[k + 1][j], ops[k])
            c = eval_op(mn[i][k], mx[k + 1][j], ops[k])
            d = eval_op(mn[i][k], mn[k + 1][j], ops[k])

            min_val = min(min_val, a, b, c, d)
            max_val = max(max_val, a, b, c, d)

        mn[i][j] = min_val
        mx[i][j] = max_val

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(mx[0][n - 1]) + "\n")

solve()

```

Текстовое объяснение решения: Динамическое программирование на отрезках. Так как присутствуют вычитание и умножение, перемножение двух отрицательных чисел может дать глобальный максимум. Поэтому для каждого подотрезка $[i, j]$ вычисляются и сохраняются как локальный максимум mx , так и локальный минимум mn .

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	1+5	6
< 1 мс	< 1 Мб	5-8+7*4-8+9	200
< 1 мс	< 1 Мб	1+2*3	9

< 1 мс	< 1 Мб	9-1-1	9
--------	--------	-------	---

Задача №15. Скобочная последовательность

Текст задачи:

Дана строка, составленная из круглых, квадратных и фигурных скобок. Определите, какое наименьшее количество символов необходимо удалить из этой строки, чтобы оставшиеся символы образовывали правильную скобочную последовательность

Код:

```
def is_pair(a, b):
    return (a == "(" and b == ")") or (a == "[" and b == "]") or (a == "{" and b == "}")

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        s = f.read().strip()

    n = len(s)

    if n == 0:
        with open("output.txt", "w", encoding="utf-8") as f:
            f.write("\n")
        return

    dp = [[0] * n for _ in range(n)]
    split = [[-1] * n for _ in range(n)]

    for length in range(1, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1

            dp[l][r] = 10**9

            if l == r:
                dp[l][r] = 1
                continue

            if is_pair(s[l], s[r]):
```

```

dp[l][r] = dp[l + 1][r - 1] if l + 1 <= r - 1 else 0
split[l][r] = -2

```

```

for k in range(l, r):
    if dp[l][k] + dp[k + 1][r] < dp[l][r]:
        dp[l][r] = dp[l][k] + dp[k + 1][r]
        split[l][r] = k

```

```

def build(l, r):
    if l > r:
        return ""
    if l == r:
        return s[l]
    if split[l][r] == -2:
        return s[l] + build(l + 1, r - 1) + s[r]
    k = split[l][r]
    return build(l, k) + build(k + 1, r)

```

```

ans = build(0, n - 1)

```

```

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(ans + "\n")

```

```

solve()

```

Текстовое объяснение решения: ДП на отрезках. Ищется минимальное число скобок для удаления, чтобы сделать строку правильной. Матрица $dp[l][r]$ хранит минимальное количество удалений на подотрезке от l до r . Ответ восстанавливается рекурсивной функцией на основе точек разреза из матрицы `split`.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	(D)	[]
< 1 мс	< 1 Мб	(())	(())
< 1 мс	< 1 Мб	((((	(пустая строка)

< 1 мс	< 1 Мб	{{O}}	{{O}}
--------	--------	-------	-------

Задача №17. Ход конем

Текст задачи:

Шахматная ассоциация решила оснастить всех своих сотрудников такими телефонными номерами, которые бы набирались на кнопочном телефоне ходом коня. Например, ходом коня набирается телефон 340-49-27. При этом телефонный номер не может начинаться ни с цифры 0, ни с цифры 8.

Код:

MOD = 10**9

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        content = f.read().strip()

    if not content:
        return

    N = int(content)

    moves = {
        0: [4, 6],
        1: [6, 8],
        2: [7, 9],
        3: [4, 8],
        4: [0, 3, 9],
        5: [],
        6: [0, 1, 7],
        7: [2, 6],
        8: [1, 3],
        9: [2, 4],
    }
```

```

dp = [0] * 10

for d in range(10):
    if d != 0 and d != 8:
        dp[d] = 1

for _ in range(2, N + 1):
    new_dp = [0] * 10
    for d in range(10):
        for to in moves[d]:
            new_dp[to] = (new_dp[to] + dp[d]) % MOD
    dp = new_dp

ans = sum(dp) % MOD

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(ans) + "\n")

solve()

```

Текстовое объяснение решения: Динамическое программирование. Состояние `dp[i]` содержит количество вариантов набора номеров, оканчивающихся на заданную цифру. На каждом шаге к текущему значению прибавляются значения из тех кнопок, с которых конь может прыгнуть на целевую цифру.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	1	8
< 1 мс	< 1 Мб	2	16
< 1 мс	< 1 Мб	3	36

Задача №18. Кафе

Текст задачи:

Около университета недавно открылось новое кафе, в котором действует

следующая система скидок: при каждой покупке более чем на 100 рублей покупатель получает купон, дающий право на один бесплатный обед (при покупке на сумму 100 рублей и меньше такой купон покупатель не получает). Однажды вам на глаза попался прейскурант на ближайшие n дней. Внимательно его изучив, вы решили, что будете обедать в этом кафе все n дней, причем каждый день вы будете покупать в кафе ровно один обед. Однако стипендия у вас небольшая, и поэтому вы хотите по максимуму использовать предоставляемую систему скидок так, чтобы ваши суммарные затраты были минимальны. Требуется найти минимально возможную суммарную стоимость обедов и номера дней, в которые вам следует воспользоваться купонами

Код:

```
INF = 10**18
```

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = list(map(int, f.read().split()))

    if not data:
        return

    n = data[0]
    prices = [0] + data[1:1 + n]

    dp = [[INF] * (n + 2) for _ in range(n + 1)]
    parent = [[None] * (n + 2) for _ in range(n + 1)]

    dp[0][0] = 0

    for i in range(1, n + 1):
        price = prices[i]
        for j in range(n + 1):
            if price > 100:
                if j > 0 and dp[i - 1][j - 1] + price < dp[i][j]:
                    dp[i][j] = dp[i - 1][j - 1] + price
                    parent[i][j] = (j - 1, False)
            else:
                if dp[i - 1][j] + price < dp[i][j]:
                    dp[i][j] = dp[i - 1][j] + price
                    parent[i][j] = (j, False)

        if j + 1 <= n and dp[i - 1][j + 1] < dp[i][j]:
            dp[i][j] = dp[i - 1][j + 1]
```

```

        parent[i][j] = (j + 1, True)

best_cost = INF
best_j = 0
for j in range(n + 1):
    if dp[n][j] < best_cost:
        best_cost = dp[n][j]
        best_j = j
    elif dp[n][j] == best_cost and j > best_j:
        best_j = j

used_days = []
cur_j = best_j
for i in range(n, 0, -1):
    prev_j, used_coupon = parent[i][cur_j]
    if used_coupon:
        used_days.append(i)
    cur_j = prev_j

used_days.reverse()

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(str(best_cost) + "\n")
    f.write(f'{best_j} {len(used_days)}\n')
    for day in used_days:
        f.write(str(day) + "\n")

solve()

```

Текстовое объяснение решения: Двумерное ДП, где состояние $dp[i][j]$ показывает минимальные затраты к i -му дню с остатком из j купонов. Реализованы переходы для покупок обедов дороже 100 рублей (с получением купона) и дешевле, а также для использования ранее накопленных купонов.

Таблица тестирования:

время	память	input	output
< 1 мс	< 1 Мб	5 110 40	260 0 2 3

		120 110 60	5
< 1 мс	< 1 Мб	3 110 110 110	220 1 1 2
< 1 мс	< 1 Мб	4 50 60 70 80	260 0 0
< 1 мс	< 1 Мб	3 200 200 200	400 1 1 2

Вывод по работе

В ходе выполнения лабораторной работы были успешно освоены и применены на практике два фундаментальных алгоритмических подхода: жадные алгоритмы и динамическое программирование.

Решение задач базового уровня с помощью жадных алгоритмов (задачи №1, 2, 3, 7, 8, 9, 10) продемонстрировало высокую эффективность методов локальной оптимизации. Были реализованы и обоснованы различные стратегии выбора: сортировка по удельной стоимости (задача о дробном рюкзаке), сортировка по времени окончания для максимизации вместимости (расписание лекций), а также сложные комбинированные стратегии с разделением объектов на группы положительного и отрицательного влияния (задача с яблоками).

Блок задач на динамическое программирование (№12, 13, 14, 15, 17, 18) позволил закрепить навыки поиска оптимальной подструктуры и работы с перекрывающимися подзадачами. Были успешно реализованы:

1. Классические алгоритмы одномерного ДП (разбиение множеств на кучи, расчет путей коня).
2. Двумерное ДП с отслеживанием дополнительных параметров состояния (задача о кафе с накоплением купонов).

3. Динамическое программирование на подотрезках (поиск максимума арифметического выражения с учетом изменения знаков и удаление минимального числа скобок в последовательности).

Для задач, требующих вывода не только оптимального значения, но и самого пути решения (например, номера дней в кафе или расстановка скобок), был успешно применен метод восстановления ответа с помощью вспомогательных массивов переходов (parent и split). Все реализованные алгоритмы уложились в жесткие системные ограничения проверяющей системы по времени ($O(N \log N)$ для жадных подходов и до $O(N^3)$ для ДП на отрезках) и памяти, что подтверждает корректность выбранных подходов.